

116. Compter les secondes : un chronomètre

`images()` compte les images même quand la boucle prend du retard : c'est l'horloge fidèle.

1. Ce qu'il faut comprendre

La console affiche **60 images par seconde**. Compter les secondes, c'est donc compter les images par paquets de 60. Mais **qui** compte ?

Si l'on écrit `compte++` à chaque tour de boucle, on se trompe dès qu'un tour dure **plus d'une image** — un gros dessin ou un long calcul suffisent. Le chronomètre retarderait alors, sans prévenir.

Ce qui est nouveau ici : `images()`, **l'horloge de la console**. Elle compte les images depuis l'allumage, **toute seule**, même quand la boucle est en retard. Elle tient dans un octet : après 255, elle repart à 0.

On retient dans `top` le moment de la dernière seconde. `ecoule = images() - top` dit combien d'images ont passé depuis. Quand il atteint 60, une seconde est écoulée : `sec++`, et `top = top + 60` avance le repère d'**exactement** une seconde. On n'écrit pas `top = images()` : si le tour avait pris une image de retard, on la perdrait, et le chrono dériverait.

Le passage de 255 à 0 ne gêne pas : un octet déborde aussi dans la soustraction. Si `top` vaut 250 et `images()` vaut 54 (il a fait le tour), `54 - 250` donne 60 : $54 + 256 - 250$. C'est la règle de l'octet (chapitre 1), qui travaille enfin pour nous.

Les minutes suivent la même idée : à 60 secondes, `sec` revient à 0 et `min` augmente. On n'écrit l'heure **qu'une fois par seconde**, quand elle change. A remet tout à zéro.

À toi : ajoute un bouton B qui met le chrono en pause (indice : une variable `marche`, et on ne compte que si elle vaut 1 — sans oublier de recalculer `top` à la reprise).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **801 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	58 o	← la console, à l'allumage — la mise en route de la console
afficher()	21 o	← main() — ta fonction
main()	114 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case

```

EcrireTexte      10 o ← texte()  – écrit un texte, lettre par lettre
EcrireNombre     33 o ← nombre() – écrit un nombre en chiffres
CopierTuiles     103 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte     14 o ← démarrage – vide les deux cartes de l'écran
LireManette      37 o ← bouton() – lit les huit boutons
Tuiles           320 o ← CopierTuiles – 20 lettres sur 43
"CHRONOMETRE"    11 o ← texte()  – un texte gravé
":"              1 o ← texte()  – un texte gravé
"A: REMETTRE A 0" 15 o ← texte() – un texte gravé

```

Lettres gravées : ACEHMNORT0123456789:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```

texte()    x 3    33 o sur place, dans main() + EcrireTexte (10 o)
nombre()   x 2    20 o sur place, dans afficher() + EcrireNombre (33 o)
images()   x 3     6 o sur place, dans main()
bouton()   x 1     5 o sur place, dans main() + LireManette (37 o)
image()    x 1     3 o sur place, dans main() + AttendreImage (28 o)

```

Tes fonctions appelées :

```

afficher() x 3     9 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : un chronomètre minutes : secondes. A le remet à 00:00.
//
// Ce qui est nouveau : mesurer le temps avec images().
// images() compte les images depuis le démarrage (60 par seconde).
// On note un repère « top » ; images() - top = les images écoulées depuis.
// Même quand images() repasse de 255 à 0, la soustraction reste juste
// (ex. : top = 250, images() = 4 → 4 - 250 donne 10 dans un uint8_t).

#include <nombre> // écrit un nombre en chiffres
#include <texte>   // écrit un texte à l'écran
#include <bouton>  // lit un bouton de la manette

```

2. UNE VARIABLE — TOP

```

uint8_t top = 0; // images() au début de la seconde en cours

```

3. UNE VARIABLE — SEC

```

uint8_t sec = 0; // les secondes (0 à 59)

```

4. UNE VARIABLE — MIN

```

uint8_t min = 0; // les minutes

```

5. UNE FONCTION — AFFICHER()

```

void afficher() { // Écrit « MM:SS » au milieu de l'écran.
    nombre(7, 6, min, 2); // 2 chiffres : 00 à 99
    nombre(10, 6, sec, 2);
}

```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    texte(4, 3, "CHRONOMETRE");
    texte(9, 6, ":");                       // Les deux-points, entre les minutes et les secondes.
    texte(3, 12, "A: REMETTRE A 0");
    afficher();                             // 00:00
    top = images();                         // le départ : maintenant

    while (true) {                          // La boucle du jeu :
        image();

        uint8_t ecole = images() - top;    // les images depuis le début de la seconde
        if (ecole >= 60) {                  // 60 images : une seconde
            top = top + 60;                 // le repère avance d'exactlyement une seconde
                                           // (pas « top = images() » : on perdrait les
                                           // images en trop, et le chrono prendrait du retard)

            sec++;
            if (sec == 60) {                // 60 secondes : une minute
                sec = 0;
                min++;
            }
            afficher();                     // on n'écrit que quand ça change
        }

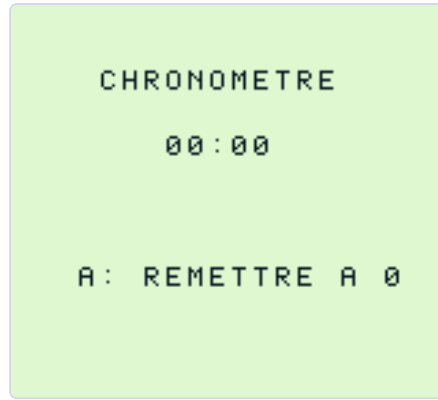
        if (bouton(A)) {                   // A : tout à zéro.
            sec = 0;
            min = 0;
            top = images();                 // nouveau départ : maintenant
            afficher();
        }
    }
}
```

3. Ce qui se passe, étape par étape

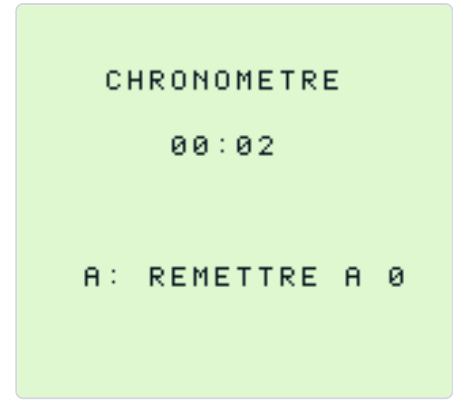
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



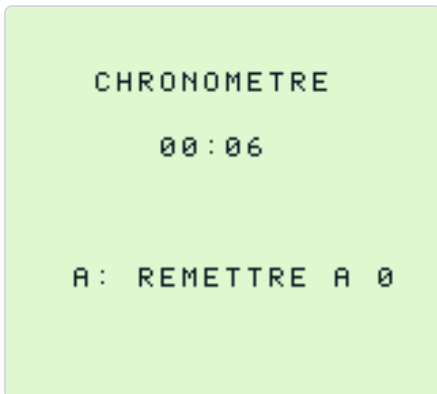
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



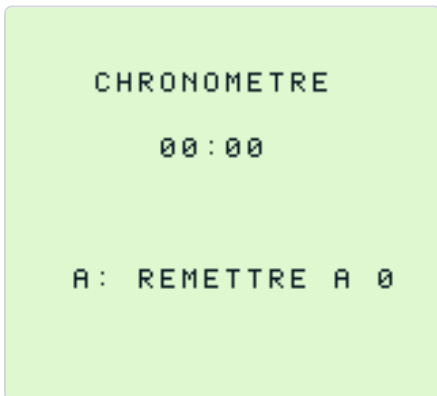
au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

Un chronomètre 00:00 qui avance d'une seconde par seconde ; A le remet à zéro.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ 600 images plus tard : 10 secondes (10)

✓ l'écran dit la même chose (10)

✓ A remet à zéro

✓ puis il repart : 2 secondes après 125 images (2)

Ensuite : **117. Un compte à rebours** — chapitre 19 : Le hasard et le temps. · [gameboy3](#) — écrire une cartouche Game Boy en C++